03

# Das Web und seine Protokolle

URL · DNS · TCP und TLS · HTTP · Caching

Web-Applikationen · Bachelor Informatik · HTWG Konstanz · WS 2026/27 · Grigorik, High Performance Browser Networking · MDN: HTTP

**Lernziele**

- ✓ Den Weg einer Anfrage von der URL bis zur Antwort in sechs Stationen beschreiben.
- ✓ Eine URL in ihre Teile zerlegen und begründen, welche Teile beim Server ankommen.
- ✓ Erklären, wie DNS einen Namen in eine IP-Adresse auflöst und warum dabei Caching und TTL entscheidend sind.
- ✓ Beschreiben, was TCP und TLS leisten und was ein Zertifikat beweist.
- ✓ HTTP-Anfragen und -Antworten lesen: Methoden, Statuscodes, Header, Redirects.
- ✓ Erklären, wie HTTP-Caching mit `Cache-Control`, ETag und `304 Not Modified` funktioniert.

## Worum es heute geht

Ihr tippt eine Adresse in den Browser, und einen Augenblick später steht die Seite da. Dazwischen passiert erstaunlich viel: Der Browser muss herausfinden, welcher Rechner gemeint ist, eine Verbindung aufbauen, sie absichern, eine Anfrage schicken und die Antwort verstehen. Beim zweiten Besuch versucht er außerdem, so wenig wie möglich erneut zu laden.

Dieses Kapitel geht diesen Weg einmal vollständig ab, an einem einzigen Beispiel: der Campus-Hackathon-Seite aus VL 2. Jeder Abschnitt ist eine Station.

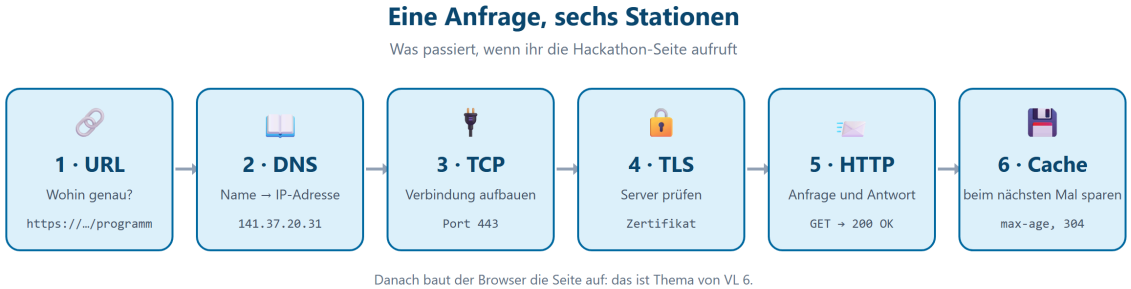



Abbildung 1: Der rote Faden dieses Kapitels: eine Anfrage, sechs Stationen.

Warum lohnt sich das, wenn am Ende ohnehin ein Framework die Arbeit macht? Weil diese Protokolle seit Jahrzehnten stabil sind. React, Vue oder das nächste Framework ändern nichts daran, dass jede Seite und jede API-Anfrage genau diesen Weg nimmt. Wer ihn kennt, findet Fehler schneller: Liegt es am Namen, an der Verbindung, am Statuscode oder am Cache?

> ★ Das **Internet** ist das Netz, das Rechner verbindet. Das **Web** ist eine Anwendung darauf, wie E-Mail auch. Es besteht aus drei Ideen, die Tim Berners-Lee 1989 am CERN zusammengebracht hat: **URLs** zum Adressieren, **HTTP** zum Übertragen und **HTML** zum Beschreiben von Dokumenten.

## Station 1: die URL

Eine URL sagt genau, welche Ressource ihr haben wollt und wie man sie bekommt:

```
https://hackathon.example.de:443/programm?tag=2#workshops
└──┘   └──────────────────┘ └┘ └──────┘ └──┘ └────────┘
Schema         Host        Port  Pfad   Query  Fragment
```

**Schema** das Protokoll, heute fast immer `https`.

**Host** der Name des Servers. Ihn löst gleich DNS auf.

**Port** meist weggelassen. Standard ist 443 für HTTPS und 80 für HTTP.

**Pfad** welche Ressource auf diesem Server gemeint ist.

**Query** Parameter nach dem `?`, als Paare aus Schlüssel und Wert, getrennt durch `&`.

**Fragment** eine Sprungmarke in der Seite nach dem `#`.

> ⚠ **Achtung:** Das Fragment wird **nicht** an den Server geschickt. Nur der Browser wertet es aus. Wer sich wundert, warum der Server einen Parameter nie sieht: Steht er hinter dem `#`?

Für euer Projekt ist die URL mehr als eine technische Adresse. Sie ist Teil der Oberfläche. Gute URLs sind lesbar, stabil und beschreiben die Ressource, nicht die Technik dahinter: `/events/42/anmeldung` ist besser als `/page.php?id=42&action=3`. Genau das prüft in M1 das Kriterium „klare URL-Struktur“.

# Station 2: DNS

Rechner im Internet finden sich über **IP-Adressen** wie `141.37.20.31`. Menschen merken sich aber Namen. Das **Domain Name System** (DNS) übersetzt das eine ins andere. Bevor der Browser auch nur ein Byte HTTP schickt, muss diese Übersetzung stehen.

## Die Auflösung Schritt für Schritt

DNS ist kein zentrales Verzeichnis, sondern eine Hierarchie. Niemand kennt alle Namen, aber jeder weiß, wen man als Nächstes fragt:

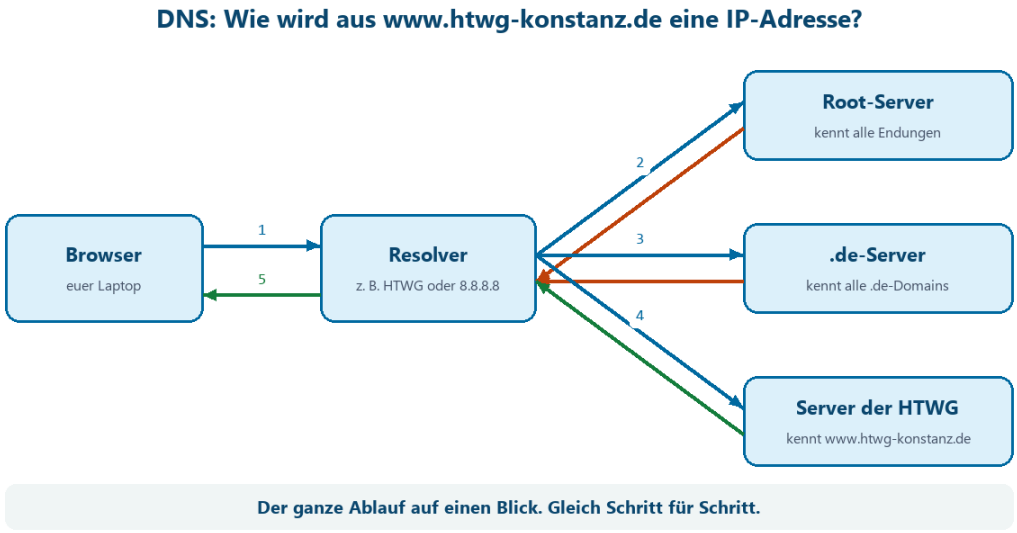



Abbildung 2: Der Resolver fragt sich durch die Hierarchie. In den Folien gibt es dazu eine Animation.

1. Der Browser schaut in seinen eigenen Cache und in den des Betriebssystems. Kein Treffer? Dann fragt er einen **Resolver**, meist den des Netzes, in dem ihr gerade seid.
2. Der Resolver fragt einen **Root-Server**. Der kennt nicht die Antwort, aber die Server für `.de`.
3. Der `.de`-Server kennt die **Nameserver** der HTWG.
4. Der Nameserver der HTWG liefert die Adresse.
5. Die Antwort geht zurück. Resolver und Browser merken sie sich.

## TTL: warum DNS trotzdem schnell ist

Jede Antwort hat eine **Time to Live** (TTL), zum Beispiel 3600 Sekunden. So lange darf sie aus dem Cache beantwortet werden. Die lange Kette läuft deshalb nur selten. Die Kehrseite: Zieht ein Server auf eine neue IP um, sehen manche Nutzer die alte Adresse noch bis zum Ablauf der TTL.

### Ein echtes Beispiel

So sieht die Antwort für die Seite der HTWG aus, abgefragt mit `nslookup` am 12.10.2026:

```
> nslookup www.htwg-konstanz.de
Name:      cms.htwg-konstanz.de
Addresses: 2001:7c0:5f0:f020::20:31
           141.37.20.31
Aliases:   www.htwg-konstanz.de
```

Darin stecken drei Record-Typen: `www.htwg-konstanz.de` ist ein **Alias** (CNAME) für `cms.htwg-konstanz.de`. Dieser Name hat eine IPv4-Adresse (**A-Record**) und eine IPv6-Adresse (**AAAA-Record**). Weitere wichtige Typen sind **MX** für Mailserver und **TXT** für Text, etwa zur Bestätigung, dass euch eine Domain gehört.

> 💡 **Tipp:** Wenn eine Seite nicht lädt, prüft zuerst den Namen: `nslookup` (Windows, macOS, Linux) oder `dig` (macOS, Linux) zeigen sofort, ob DNS eine Adresse liefert.

## Station 3 und 4: TCP und TLS

### TCP: eine zuverlässige Leitung

Mit der IP-Adresse baut der Browser eine **TCP-Verbindung** auf. TCP sorgt dafür, dass alle Daten ankommen, und zwar in der richtigen Reihenfolge. Verlorene Pakete werden neu geschickt. Dafür braucht es vorab einen **Handshake**: drei Nachrichten, bevor die erste Nutzlast fließt.

Das kostet Zeit. Liegt der Server weit weg, dauert eine Hin-und-zurück-Runde (**Round Trip**) schnell 100 Millisekunden oder mehr, und Physik lässt sich nicht beschleunigen. Grigorik nennt die Latenz deshalb den eigentlichen Engpass des Web, nicht die Bandbreite. Viele Optimierungen, die ihr in VL 14 kennenlernt, sparen genau solche Runden.

### TLS: die Leitung absichern

Bei `https` folgt auf TCP ein zweiter Handshake: **TLS** (Transport Layer Security). TLS liefert drei Garantien:

1. **Vertraulichkeit:** Niemand unterwegs kann mitlesen, weder das WLAN noch der Provider.
2. **Integrität:** Niemand kann die Daten unbemerkt verändern.
3. **Authentizität:** Der Server beweist mit einem **Zertifikat**, dass er wirklich der ist, für den er sich ausgibt.

Das Zertifikat ist von einer **Certificate Authority** (CA) unterschrieben, der euer Browser vertraut, etwa Let's Encrypt. Der Browser prüft: Stimmt der Name im Zertifikat mit dem Host überein? Ist es noch gültig? Führt die Kette der Unterschriften zu einer bekannten CA? Erst dann fließen HTTP-Daten, verschlüsselt mit einem Schlüssel, den beide Seiten im Handshake vereinbart haben.

> ⚠ **Achtung:** Das Schloss im Browser heißt nur: Die Verbindung zu **diesem** Namen ist sicher. Es heißt nicht, dass die Seite vertrauenswürdig ist. Auch eine Phishing-Seite unter `htwg-konstanz-login.de` kann ein gültiges Zertifikat haben.

Heute gilt: kein Web-Projekt ohne HTTPS. Browser markieren reine HTTP-Seiten als „nicht sicher", und viele moderne Browser-Funktionen gibt es nur über HTTPS. Hosting-Plattformen richten das Zertifikat automatisch ein.

## Station 5: HTTP

Jetzt endlich spricht der Browser HTTP. Das **Hypertext Transfer Protocol** ist ein einfaches, textbasiertes Frage-Antwort-Protokoll: Der Client schickt eine **Anfrage** (Request), der Server eine **Antwort** (Response). Und HTTP ist **zustandslos**: Jede Anfrage steht für sich, der Server erinnert sich von sich aus an nichts. Wie man trotzdem Logins und Warenkörbe baut, ist Thema von VL 10.

### Anfrage und Antwort lesen

Mit `curl -v` seht ihr HTTP ohne Browser. So sieht der Aufruf der Hackathon-Seite auf einem lokalen Server aus:

```
> GET /03_gestylt.html HTTP/1.1
> Host: localhost:8765
> Accept: */*
>
< HTTP/1.0 200 OK
< Content-type: text/html
< Content-Length: 2093
< Last-Modified: Wed, 30 Sep 2026 17:03:10 GMT
```

Die Zeilen mit `>` sind die Anfrage: zuerst die **Request-Zeile** mit Methode, Pfad und Version, dann **Header**, dann eine Leerzeile, danach bei Bedarf ein **Body**. Die Zeilen mit `<` sind die Antwort: **Statuszeile** mit Code, dann Header, dann der Body, hier das HTML.

### Methoden

Die Methode sagt, was mit der Ressource passieren soll:

| Methode | Bedeutung | sicher | idempotent |
|---|---|---|---|
| `GET` | Ressource lesen | ja | ja |
| `POST` | etwas Neues anlegen oder eine Aktion auslösen | nein | nein |
| `PUT` | Ressource vollständig ersetzen | nein | ja |
| `PATCH` | Ressource teilweise ändern | nein | nicht garantiert |
| `DELETE` | Ressource löschen | nein | ja |

**Sicher** heißt: Die Anfrage ändert nichts auf dem Server. **Idempotent** heißt: Zweimal ausführen hat dieselbe Wirkung wie einmal. `DELETE /events/42` löscht beim zweiten Mal nichts mehr, `POST /anmeldungen` legt dagegen jedes Mal eine neue Anmeldung an. Das ist wichtig, weil Browser, Proxies und Bibliotheken idempotente Anfragen bei Netzproblemen einfach wiederholen dürfen.

⚠ **Achtung:** Ein `GET` darf nie etwas ändern. Ein Link wie `/events/42/loeschen` ist ein Fehler: Suchmaschinen, Vorschau-Funktionen in Chat-Apps und Browser, die Seiten vorab laden, folgen Links ungefragt.

## Statuscodes

Die erste Ziffer verrät die Klasse:

**2xx Erfolg** `200 OK`, `201 Created` nach dem Anlegen, `204 No Content` ohne Body.

**3xx Umleitung** `301 Moved Permanently`, `302 Found`, `304 Not Modified` beim Caching.

**4xx Fehler des Clients** `400 Bad Request`, `401 Unauthorized`, `403 Forbidden`, `404 Not Found`.

**5xx Fehler des Servers** `500 Internal Server Error`, `503 Service Unavailable`.

⚠ **Achtung: 401 oder 403?** `401` heißt: Wer bist du? Es fehlt eine gültige Anmeldung. `403` heißt: Ich weiß, wer du bist, aber du darfst das nicht. Wer angemeldet ist, aber kein Admin, bekommt also `403`. Diese Unterscheidung braucht ihr in M3 bei den Rollen.

## Header

Header sind Metadaten als Paare aus Name und Wert. Ein paar braucht ihr ständig:

`Host` welche Domain gemeint ist. Pflicht, weil viele Domains auf einem Server liegen können.

`Accept`, `Content-Type` welches Format der Client möchte und welches der Body tatsächlich hat, etwa `text/html` oder `application/json`.

`Location` das Ziel einer Umleitung.

| | |
|---|---|
| `Cache-Control`, `ETag`, `Last-Modified` | steuern das Caching, gleich mehr dazu. |
| `Authorization`, `Cookie`, `Set-Cookie` | Anmeldung und Zustand, Thema von VL 10 und 11. |

### Umleitungen

Ruft man die HTWG-Seite über `http://` auf, antwortet der Server so:

```
HTTP/1.1 302 Found
Location: https://www.htwg-konstanz.de/
```

Der Browser folgt dem `Location`-Header automatisch und stellt eine zweite Anfrage, diesmal über HTTPS. Moderne Browser gehen sogar noch einen Schritt weiter: Chrome und Firefox probieren bei einer `http`-Adresse von sich aus zuerst HTTPS. Im Network-Tab erscheint dann eine interne Umleitung mit `307`, die nie beim Server war. Die echte Antwort des Servers seht ihr nur ohne Browser, etwa mit `curl`. `302` bedeutet „vorübergehend woanders“, `301` bedeutet „dauerhaft umgezogen“. Bei `301` merkt sich der Browser die neue Adresse und fragt beim nächsten Mal gar nicht erst die alte. Für die Umleitung von HTTP auf HTTPS wäre `301` deshalb die bessere Wahl. Noch einen Schritt weiter geht der Header `Strict-Transport-Security`: Damit verspricht der Server, dass er nur noch über HTTPS erreichbar ist.

Umleitungen lösen auch ein klassisches Formularproblem, das **Post/Redirect/Get-Muster**: Nach einem erfolgreichen `POST` antwortet der Server nicht mit einer Seite, sondern mit einer Umleitung auf eine Ergebnisseite. Lädt jemand diese Seite neu, wird nur ein harmloses `GET` wiederholt, keine doppelte Anmeldung.

## Station 6: Caching

Die schnellste Anfrage ist die, die gar nicht erst stattfindet. Deshalb dürfen Browser Antworten aufheben und wiederverwenden. Was sie aufheben dürfen und wie lange, bestimmt der Server mit Headern.

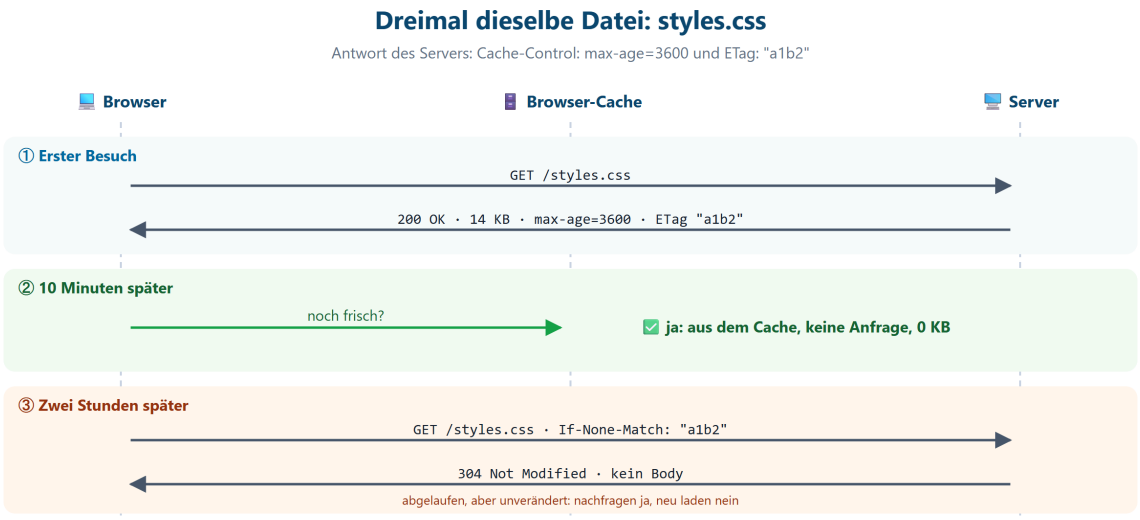



Abbildung 3: Dreimal dieselbe Datei: einmal laden, einmal aus dem Cache, einmal nur nachfragen.

## Wie lange ist etwas frisch?

Der wichtigste Header ist `Cache-Control`:

`max-age=3600` Die Antwort ist 3600 Sekunden lang frisch. So lange fragt der Browser gar nicht nach.

`no-cache` Der Browser darf speichern, muss aber vor jeder Verwendung beim Server nachfragen.

`no-store` gar nicht speichern. Richtig für Seiten mit persönlichen Daten.

`private` und `public` nur der Browser darf speichern, oder auch Zwischenstationen wie ein CDN.

## Nachfragen statt neu laden

Ist eine Antwort abgelaufen, muss sie nicht neu geladen werden. Der Browser fragt nach, ob sie sich geändert hat. Dafür hat er beim ersten Mal einen **Validator** bekommen: einen `ETag`, eine Art Fingerabdruck der Datei, oder ein `Last-Modified`-Datum. Bei der nächsten Anfrage schickt er ihn mit (`If-None-Match` oder `If-Modified-Since`). Ist die Datei unverändert, antwortet der Server mit `304 Not Modified`, ohne Body.

Das lässt sich auch lokal zeigen. Python liefert die Hackathon-Seite mit `Last-Modified` aus. Fragt man mit genau diesem Datum nach, kommt `304`:

```
curl -H "If-Modified-Since: Wed, 30 Sep 2026 17:03:10 GMT" \
     http://localhost:8765/03_gestylt.html     → 304
```

### Lange cachen, trotzdem aktuell: Hashes im Dateinamen

Für CSS, JavaScript und Bilder wäre ein sehr langes `max-age` ideal. Aber was, wenn ihr das CSS ändert? Die Lösung: Der Dateiname enthält einen Fingerabdruck des Inhalts. Ändert sich der Inhalt, ändert sich der Name, und für den Browser ist das eine neue Datei. Die HTWG macht genau das:

```
GET /typo3temp/assets/compressed/merged-7c99adcc….css
Cache-Control: max-age=31536000
```

Die CSS-Datei darf ein Jahr lang im Cache bleiben. Die HTML-Seite selbst kommt dagegen mit `Cache-Control: private, no-store`, damit Änderungen und die neuen Dateinamen sofort ankommen. Build-Werkzeuge wie Vite erzeugen solche Dateinamen automatisch. Ihr seht das spätestens in M2.

★ HTML kurz oder gar nicht cachen, damit Änderungen sofort ankommen. Dateien mit Hash im Namen sehr lange cachen, weil sich ihr Inhalt nie ändert. Alles dazwischen mit `ETag` oder `Last-Modified` absichern, damit der Server mit `304` antworten kann.

## Ausblick: HTTP/1.1, HTTP/2 und HTTP/3

HTTP gibt es in mehreren Versionen, die sich vor allem darin unterscheiden, wie sie die Verbindung nutzen. Mit HTTP/1.1 läuft pro Verbindung eine Anfrage nach der anderen, Browser öffnen deshalb mehrere Verbindungen parallel. HTTP/2 schickt viele Anfragen gleichzeitig über eine Verbindung. HTTP/3 baut statt auf TCP auf dem neueren Protokoll QUIC auf und spart dadurch Round Trips beim Verbindungsaufbau. An Methoden, Statuscodes und Headern ändert sich nichts. Welche Version eine Seite nutzt, zeigt die Spalte **Protocol** im Network-Tab. Mehr dazu in VL 14.

## Werkzeuge

💡 **Tipp:**

- **DevTools, Tab Network** ( `F12` ): jede Anfrage mit Methode, Status, Größe, Zeit und allen Headern. Mit `Strg+Shift+R` lädt ihr die Seite ohne Cache neu, die Spalte **Size** zeigt bei Treffern „(memory cache)“ oder „(disk cache)“.
- **curl** im Terminal: `curl -v https://…` zeigt Anfrage und Antwort vollständig, `curl -I` nur die Header. In der Windows-PowerShell schreibt ihr `curl.exe`, weil `curl` dort ein anderes Programm aufruft.
- **nslookup** oder **dig**: DNS-Auflösung prüfen.

## Was ihr davon für M1 braucht

- **URL-Struktur:** Plant eure Seiten als Ressourcen mit sprechenden Pfaden. Verlinkt relativ, damit der Prototyp lokal und später auf einem Server gleich funktioniert.
- **Lokal ausliefern statt Datei öffnen:** Startet euren Prototyp mit einem kleinen Server ( `npx serve` oder `python -m http.server` ). Dann seht ihr im Network-Tab echte Anfragen, Statuscodes und 404-Fehler für fehlende Dateien.
- **Kein 404 übersehen:** Ein vertippter Pfad zu CSS oder Bild fällt im Browser oft nicht auf, im Network-Tab sofort.

## Typische Missverständnisse

⚠️ **Achtung:**

- „Internet und Web sind dasselbe.“ Das Web ist eine Anwendung auf dem Internet.
- „Der Server sieht alles nach dem `#`.“ Das Fragment bleibt im Browser.
- „HTTPS heißt, die Seite ist seriös.“ Es heißt nur, dass die Verbindung zu diesem Namen sicher ist.
- „`no-cache` heißt, nichts wird gespeichert.“ Es heißt: vor jeder Verwendung nachfragen. Nichts speichern heißt `no-store`.
- „`401` und `403` sind dasselbe.“ `401` : nicht angemeldet. `403` : angemeldet, aber nicht berechtigt.

## Zusammenfassung

★ Jede Anfrage im Web nimmt denselben Weg. Die **URL** sagt, welche Ressource gemeint ist. **DNS** löst den Namen über eine Hierarchie von Servern in eine IP-Adresse auf und cacht die Antwort für die TTL. **TCP** baut eine zuverlässige Verbindung auf, **TLS** verschlüsselt sie und beweist mit einem Zertifikat, wer der Server ist. **HTTP** ist ein zustandsloses Frage-Antwort-Protokoll aus Methode, Pfad, Headern und Body, die Antwort trägt einen Statuscode. **Caching** spart Anfragen: `Cache-Control` legt fest, wie lange etwas frisch ist, `ETag` und `304` sparen das erneute Laden, und Hashes im Dateinamen erlauben sehr lange Cache-Zeiten.

## Kontrollfragen

1. Welche Teile von `https://shop.example.de/produkte?farbe=rot#bewertungen` kommen beim Server an, welche nicht?
2. Ein Server zieht auf eine neue IP-Adresse um. Manche Nutzer erreichen noch stundenlang den alten Server. Woran liegt das?
3. Eine Seite zeigt ein gültiges Schloss. Was beweist das, und was nicht?
4. Ihr ruft `DELETE /events/7` zweimal auf. Was sollte beim zweiten Mal passieren, und warum ist `DELETE` trotzdem idempotent?
5. Eure CSS-Datei hat `Cache-Control: max-age=31536000`. Ihr ändert das Design, aber Nutzer sehen weiter das alte. Was ist schiefgelaufen, und wie löst man das?

## Quellen


- Ilya Grigorik: *High Performance Browser Networking.* O'Reilly, 2013, Kapitel 1 bis 4 (Latenz, TCP, TLS) und 9 bis 12 (HTTP). Frei lesbar unter hpbn.co.
- MDN Web Docs: *An overview of HTTP*, *HTTP caching*, *HTTP response status codes.* developer.mozilla.org
- RFC 9110 *HTTP Semantics* und RFC 9111 *HTTP Caching*, IETF, 2022.
- Messungen an `www.htwg-konstanz.de` mit `nslookup` und `curl`, 12.10.2026.